

Prime Flight / RampVision — technical audit and measurement campaign

One-page digest · 6 October 2026 · Valentyn Fedorov (lead) · sources: architecture review of 7 September (docs/arch_review), 18 analysis reports (docs/analysis), 4 decision records (docs/decisions), the task board (52 tasks), the prime_flight repository. Every figure is measured unless marked as a plan.

Verdict. The system works, but it is one pipeline copied 27 times, and about half of the checks verify a proxy for the behaviour rather than the behaviour itself. Real time is feasible with this code: 20 of the client's 30 checks already run live with the post-processing verdict, the shared General Model and tracker are ported frame by frame and proven byte-identical to production, and the production GPU keeps up once the detector heads run through TensorRT. Two things block delivery: the rule that decides which aircraft on the stand is ours (2.9 points of accuracy) and the uplink at the gate (8 Mbit/s per station minimum).

1 · The system today

DXGAT checks 30 safety items of an aircraft turnaround from two gate cameras (cone, wing; 1080p, 8 fps, H.264 at a constant 4.0 Mbit/s each). Everything is **post-processing**: the CameraBox writes 1-minute chunks → bucket → merge into a full-turn video → General Model (YOLO heads gm@1088, chocks@1280, vehicle@1088 + camera / aircraft-type / entity classifiers) → trackers (aircraft, belt loader, GSE, person) → 27 module jobs → Pass / Fail / Not observed → MongoDB → RampVision. **Time to Result ≈ 24 h**: on the median of 148 videos (87 min of recording) 20.3 h pass from recording to result — 9.5 h for the last chunk to reach the bucket, 2.5 h waiting for and running merge, GM 2.4 h (1.7× slower than recording), tracker 1.3 h, modules 0.9 h; GM + tracker = 3.7 GPU-pod hours per video. A production GPU job gets a Tesla T4 (14.6 GB), 3.9 vCPU of a 2013-generation Xeon and 12 GB of RAM; the worker has no live-stream input.

2 · What the architecture review found (static review of pinned source, baseline 7 September)

- **Scope.** 32 repositories (27 check modules, GM, tracker, cv_common, camera and worker software), 78 source files, **417 component occurrences** catalogued (379 code, 30 library, 8 flagged: missing dependencies, optional-off, stub, inactive, input gap) and distilled into **32 shared concepts E01–E33** behind the 109 inputs the checks consume; no inference was run. The 27 modules implement 30 checks: the chock module gives three verdicts, “proper belt loader approach” is derived from 3-stop + hand signals.
- **Structure.** Each module is its own repository with its own copy of the pipeline: 24 of 27 recompute the arrival stage from the tracker's private fields, 4 read the whole recording twice (aircraft-chocks, hand-signals, pin-verification, safety-zone), the modules pin **10 different revisions of cv_common**, and two (safety huddle, pre-departure walk-around) import helpers that do not exist at their pinned revision. The most duplicated concepts are the aircraft's arrival, departure and movement (E01, E02, E27), which almost every check derives for itself.
- **Semantics. 13 of the 30 checks verify a proxy**, not the behaviour the checklist names: hand signals = raised wrists; pin verification = a worker's posture near the nose wheel; “cones removed after GSE is chocked” never tests chocking; “cargo doors opened” never ties a door to its loader; conditioned-air removal = a change of area on screen; crew presence = a head count over a window; nose-wheel chock removal = sustained presence, not a removal edge. **Wing walkers with approved wands passes the wand part unconditionally** on the active code path. In 6 checks the thresholds or windows in the code differ from the client's description (a “five-minute” window is 60 s; a report that says 40 % tests 65 %; YAML counts 6/4 vs code 7/6).
- **Pipeline.** Metadata rows are joined by position, not by frame id; the tracker's private fields are serialized into the contract, so consumers break with every tracker version; GM overloads the confidence field of the aircraft row with its height; nothing carries a schema version (15–16 incompatible inference formats coexist in the bucket, visible only as a crash); the camera software's capture boundary is not the analytic arrival/departure.

3 · What was built and proven (8 September – 1 October, pf/ package, 186 tests)

- **GM v2 (pf/gm):** the three production ONNX heads once per frame with one letterbox per input size and parallel threads — rows byte-identical to the v1 sequence, 22.4 → 16.8 ms, 8.0 ms with TensorRT fp16 (tolerant parity); pair recall against production rows 99.9 % on a full turnaround; the second pass (camera type, main aircraft, frame_stopped) ported as **causal rows**, so no decision waits for the end of the video.
- **Tracker v2 (pf/tracker):** the production loop on vendored production classes, **byte-identical to the seeded production pin** (35 730 of 35 730 lines on two whole events); exact fast paths –23 %; one tracker per event (X1); T_ARR, T_DEP, BL_AT_DOOR, BL_LEAVE and pushback_attached derived causally (the production frame on 5 of 7 reference videos); the v2 bus 10.8× lighter without private fields.
- **Real-time branch (pf/rt):** GOP chunker (30 frames = 3.75 s, stream copy) → receiver with ordering, gap-fill and absolute frame_id (X2) → pinned decoder (X4) → GM v2 → causal rows → Tracker v2 → one OS process per **unchanged production module** (its detect() blocks until the frame arrives; pixels through a shared-memory ring; declared inputs cut the record from 13.5 KB to 1.6–2.6 KB) → OutputBus (verdict the moment it is produced), live page, http sink as the seam for alerts; end-of-session marker so the session length is unknown until the recording closes.
- **Harness and decisions.** Tolerant GM parity, seeded tracker parity, module-verdict gates on the 69-event test set, cost / joint-run / live-accuracy / resource-fit / uplink tools; CI on GitHub Actions in 1 min (lint, 186 tests, the branch end to end on a synthetic clip asserting X2 and X4); GPU gates written for a self-hosted runner (none yet). ADR-001 v1-compat sink + v2 bus, ADR-002 second pass in real time, ADR-003 TensorRT as a versioned runtime, ADR-004 component interfaces.

4 · What the measurements established

Question	Measured answer
Checks that run live unchanged	20 of 30 (one module each): 15 answer during the turnaround, 3 only when the camera stops (vests, GSE chocks, handrails on GSE), 2 with a small report shift; 6 checks from 4 modules read the recording twice and need rework; 3 are not hosted (both walk-arounds, hair policy); 1 is derived. Alone 19–47 ms of the 125 ms budget, the module itself under 3.3 ms for 17 of 20 (vests 18.8 ms + 5.5 cores, handrails on GSE 12.5 ms).
Verdicts live vs post	20 of 20 modules identical on the reference event (whole turnaround at real-time speed: 56 ms frame path, GPU 24.5 % busy, 4.2 cores, 18 GB RAM); 78 of 78 verdicts on the six events then on disk; 851 of 858 test-set task verdicts with v2 inputs; scoped rows change 0 of 740.
Accuracy	86.7 % post-processing → 83.8 % live. The whole gap is one rule — which aircraft on the stand is ours [“longest track so far” vs batch's “longest of the video”] — on about one turnaround in six: 36 verdicts changed on 10 of 21 events, T_arr off by up to 23 min. Two alternative rules tried live (largest-alive, size gate): 82.9 %, not a fix. Needs turnaround ownership in the stage detector: re-armable anchors + stand geometry.
Shared cost	GM by head set 8.5 / 11.5 / 14.3 / 19.2 ms (gm / +chocks / +vehicle / all three; TensorRT 11.1 for all three); tracker by classes 4.9 / 7.7 / 20.7 / 26.2 ms (airplane / +person / +belt loader / all four): the belt loader costs +15.7 ms; breakdown — belt loader path 9.6, DeepSORT 8.3, aircraft 4.0, noise 2.3, GSE 2.4.
Production pod (T4, 4 vCPU)	Ten pixel-free modules, heads emulated 3× slower: 144 ms per frame with CUDA (does not keep up) → 115 ms with TensorRT at the pessimistic end, 60 ms at the optimistic ; 2 slow cores suffice, 1 does not. The T4 is not the blocker, the four Haswell threads are. Pending: node_bench.py in a gpu-pool pod (2 min) to replace the emulation.
Cost and scale	The same work as post-processing, done as it happens: ≈1.1× GPU time for the whole set per host (2 160 vs 2 019 s), 1.0–1.5× for one module when streams share a card, 2.3–4.9× with one stream per card. 40 gates = 80 streams: ~20 cards for the light set (4 streams per 16 GB card), ~40 with pixel modules (VRAM-bound).
Station uplink	4.0 Mbit/s per camera (constant on all 21 videos), 8 per station, 60 MB per minute, 86 GB per day. Below 8 Mbit/s the queue never drains; 20 Mbit/s sustained is the target. The chunk length sets the delay: 30-frame chunks 3–5 s behind at 20 Mbit/s, 1-minute chunks 53–84 s at any speed ≥ 20. Model checked against six live runs over a throttled link (within 0.2 s).
realtime-pipeline worker (GitLab, 1 Oct)	One always-up worker, containers on one time-sliced GPU: ingest → stage detector → GM (chocks + vehicle only) → trackers (person, BL, GSE; plane track copied) → modules. Our GM v2 and tracker v2 are in it verbatim (import paths only). Its split puts the aircraft decision in the stage detector — the right place. Its JPEG-in-JSON hand-off costs ≈13.5 ms per frame here, ≈22 ms on a pod core; shared memory removes it.

5 · Plan: order of the checks, roadmap page, optimisation

Order into minutes (greedy on the marginal cost of the next check — a stream pays for a head or a tracked class once: the first belt-loader check costs +22 ms, every later one under 2 ms): the three checks already assembled first (3-stop brake, pushback pathway, pushback wing walkers — running together live, alert hooks planned), then **wave A** (15 checks, no rework, +0.1...+5.1 ms each), **wave B** (3 checks with a verdict at session end — interim-verdict hook, PF-Q2-12), **wave C** (2 after-the-fact checks — trigger agreed with the client, PF-Q3-06), **wave D** (4 checks that read the recording twice — PF-Q3-04, PF-Q2-11); hours keep hand signals, both walk-arounds and the derived belt-loader approach. Roadmap page: 24 checks in minutes, 4 in hours, 2 on the edge — hair policy M3 and **safety vests M6 instead of nose gear chocks** (the only critical check free of GM and stage; needs an accelerator in the box and a smaller detector + classifier); pace 5 by M3, 16 by M7, 24 by M12.

Optimisation levers, ranked by what each buys: (1) TensorRT heads — implemented, needs the verdict gate; (2) per-stream heads and classes — implemented, exact for GM; (3) the two tracker segmentors through ONNX/TensorRT (belt loader 9.6 + aircraft 4.0 ms); (4) one DeepSORT forward instead of three (8.3 ms); (5) skip re-id when association is unambiguous; (6) noise-estimate interval; (7) chocks/vehicle heads at a lower rate; (8) INT8 on the T4; (9) NVDEC; (10) one model instance per card. Not worth the time: the modules (17 of 20 under 3.3 ms). Every change is proved by one ladder: tolerant GM parity → seeded tracker parity + anchors → 740 test-set verdicts → live on the 21 events + accuracy.

6 · Risks, decisions, next 90 days

Risk	Measured size	What closes it
Which aircraft is ours	~2.9 pts accuracy, 36 verdicts on 10 of 21 events	Stage detector owns the turnaround: re-armable anchors + stand geometry (PF-Q2-02, PF-Q1-03)
Proxies read as verification	13 checks verify a stand-in; wands pass unconditionally	Per-check statement of what is verified, agreed with the client; fix the wand path first
Production CPU, not GPU	4 threads of a 2013 CPU; 20 checks need ~8 modern cores	TensorRT, per-stream scoping, shared-memory hand-off; node_bench in a pod
Transport from the gate	8 Mbit/s per station, 86 GB/day, measured nowhere	Sustained upload measured at one gate; below 8 Mbit/s no real time there
Dependency debt	10 cv_common pins, 2 broken imports, positional joins, no schema	Frame contract with frame_id + schema_version (X2, X3); one pin per wave
Fail labels / after-the-fact checks	3 of 4 first-queue checks have ~no fails; 4 checks decide post hoc	Paired batch↔live comparison until labels exist; trigger change with the client

Decisions needed: (1) accept the review's 32 component boundaries and close its per-occurrence decisions; (2) state what each check verifies and fix the wand path before any check is sold as real time; (3) TensorRT for the heads after the verdict gate; (4) the stage detector owns the turnaround, built in the realtime-pipeline worker, GM/tracker consumed as one package, not copied; (5) safety vests on the edge at M6, accelerator funded; (6) triggers for the four after-the-fact checks opened with the client; (7) a GPU runner, read access to production reports, one gate to measure.

Next 90 days (proposed): 30 d — review decisions closed, wand path fixed, TensorRT gate closed and default switched, pod benchmark run, the assembled trio running as real modules in the worker with batch-identical verdicts, uplink measured at one gate. 60 d — stage detector v0 with re-armable anchors, accuracy back at the batch level on the 21 events, per-check statements agreed, tracker segmentors through ONNX, shared-memory hand-off. 90 d — wave A (15 checks) in the worker on one library pin, one station live end to end, alert design handed over. No verdict gets better than post-processing gives today; what changes is when it arrives — seconds to minutes instead of a day.